

Spanning Tree

.

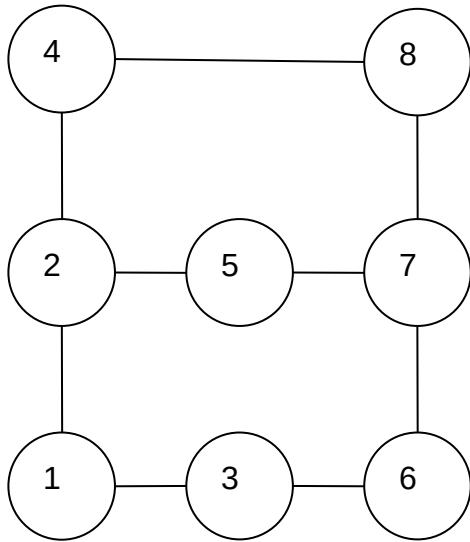
Let $G(V,E)$ be any connected graph then any subgraph $T=(V,E')$ of given graph G consist of all the vertices in G which is still connected and acyclic is called spanning tree of given graph.

Graph Traversal

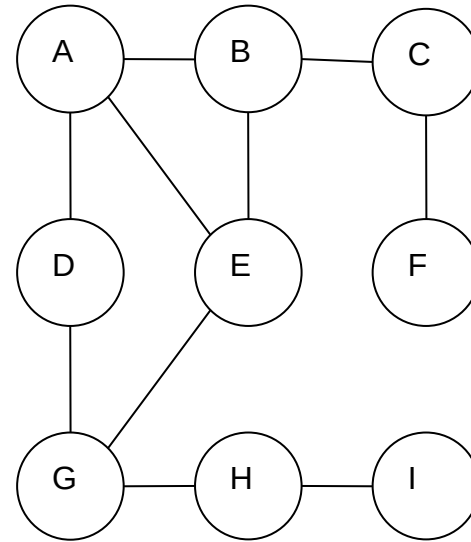
Breadth-first search:

- This is one of the simplest methods of graph searching.
- Choose some vertex arbitrarily as a root.
- Add all the vertices and edges that are incident in the root.
- The new vertices added will become the vertices at the level 1 of the BFS tree.
- Form the set of the added vertices of level 1, find other vertices, such that they are connected by edges at level 1 vertices.
- Follow the above step until all the vertices are added

- The *Breadth First Traversal* begins with a given vertex and then
- it next visits all the vertices adjacent to v , putting the vertices adjacent to these in a waiting list to be traversed after all vertices adjacent to v have visited.
- Breadth First Traversal uses queue.



Starting from 1 Breadth First Traversal yields
1, 2, 3, 4, 5, 6, 8, 7



Starting from A Breadth First Traversal yields
A, B, D, E, C, G, F, H, I

Depth First Traversal

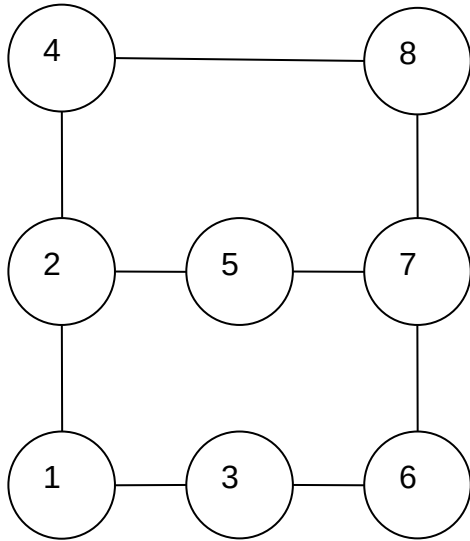
The *Depth First Traversal* algorithm is roughly analogous to preorder tree traversal, the algorithm follows:

suppose that the traversal has just visited a vertex v , and let $w_0, w_1, \dots, w_k$ be the vertices adjacent to v

next, visit w_0 and keep $w_1, \dots, w_k$ waiting

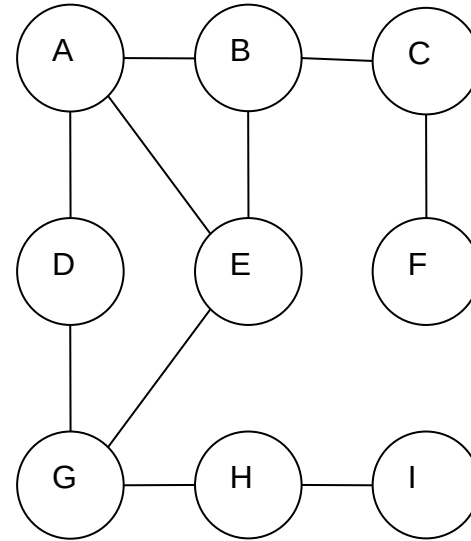
after visiting w_0 , we traverse all the vertices to which it is adjacent before returning to traverse $w_1, \dots, w_k$.

Depth first traversal uses stack to store the adjacent vertices but one.



Starting from 1 Breadth First Traversal yields

1, 2, 4, 8, 7, 5, 6, 3



Starting from A Breadth First Traversal yields

A, B, C, F, E, G, D, H, I

Adjacency list		Adjacency list	
1	2, 3	A	B, D, E
2	1, 4, 5	B	A, C, E
3	1, 6	C	B, F
4	2, 8	D	A, G
5	2, 7	E	A, B, G
6	3, 7	F	C
7	5, 6, 8	G	D, E, H
8	4, 7	H	G, I
		I	H
DF Traversal:		DF Traversal:	
	1, 2, 4, 8, 7, 5, 6, 3		A, B, C, F, E, G, D, H, I

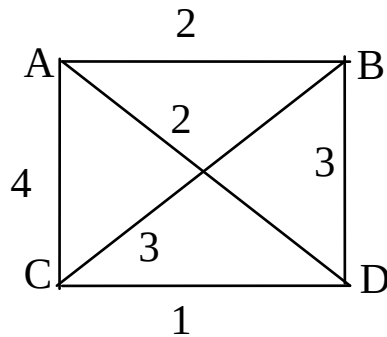
BFT :1, 2, 3, 4, 5, 6, 8, 7

BFT : A, B, D, E, C, G, F, H, I

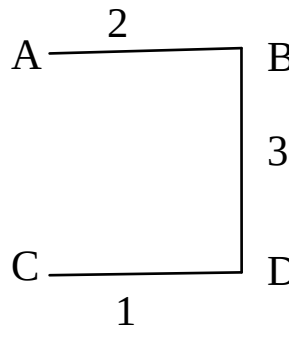
Minimum Spanning Tree

- The cost of a spanning tree of a weighted, undirected graph is the sum of the costs (weights) of the edges in tree.
- The minimum spanning tree of a weighted undirected graph is the spanning tree that connects all the vertices in G at lowest cost. So, if T is the minimum spanning tree of graph G and T' is any other spanning tree of G then,

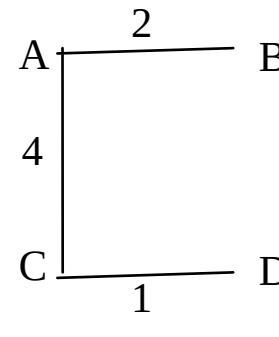
$$w(T) \leq w(T')$$



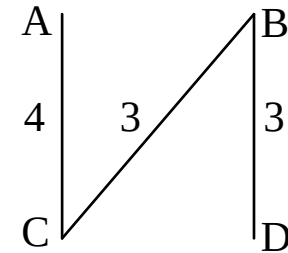
G



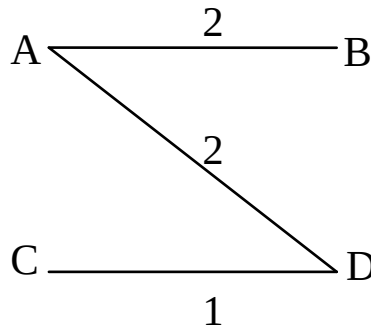
T1



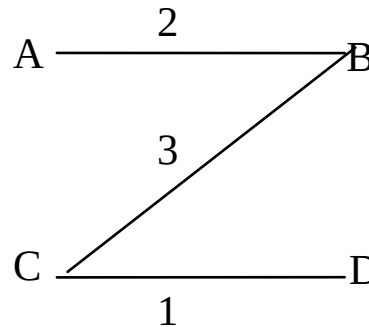
T2



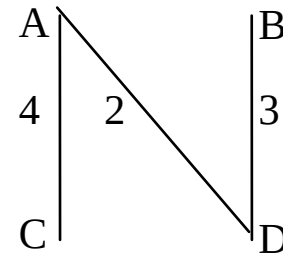
T3



T4



T5



T6

Here, T_1 to T_6 are the *spanning trees* of **G**. Among them, **T4** has minimum cost (i. e. 5). So, **T4** is the minimum spanning tree of graph G

There are several algorithms available to determine the *minimal spanning tree* of a given weighted graph.

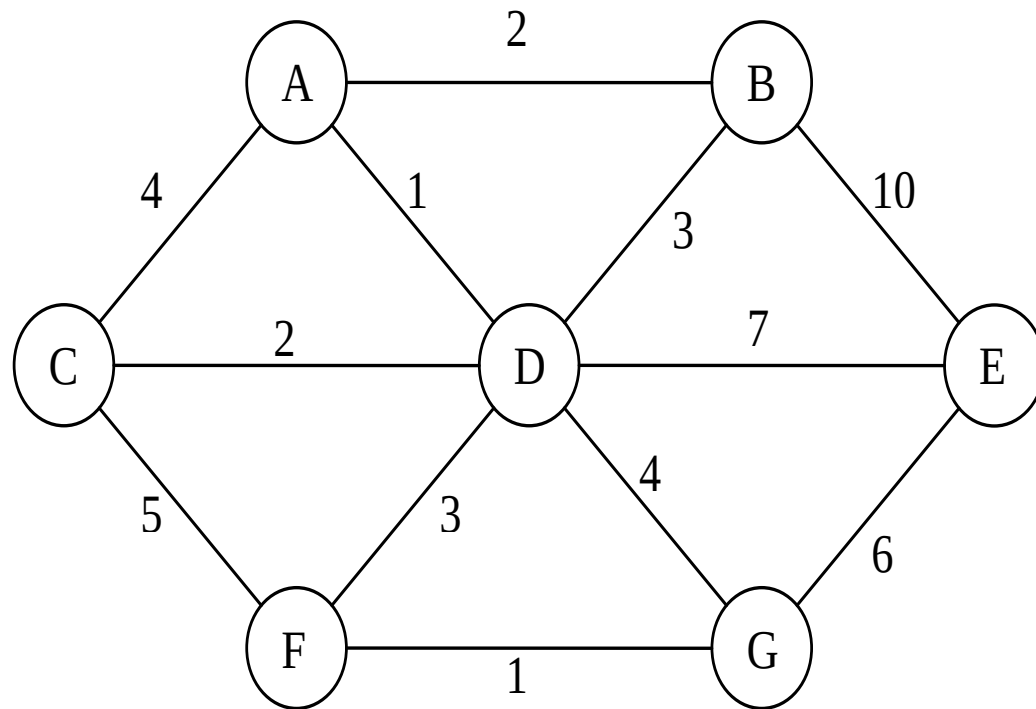
Kruskal's Algorithm

- To determine minimum spanning tree consider a graph G with n vertices.

Step 1: List all the edges of the graph G with increasing weights

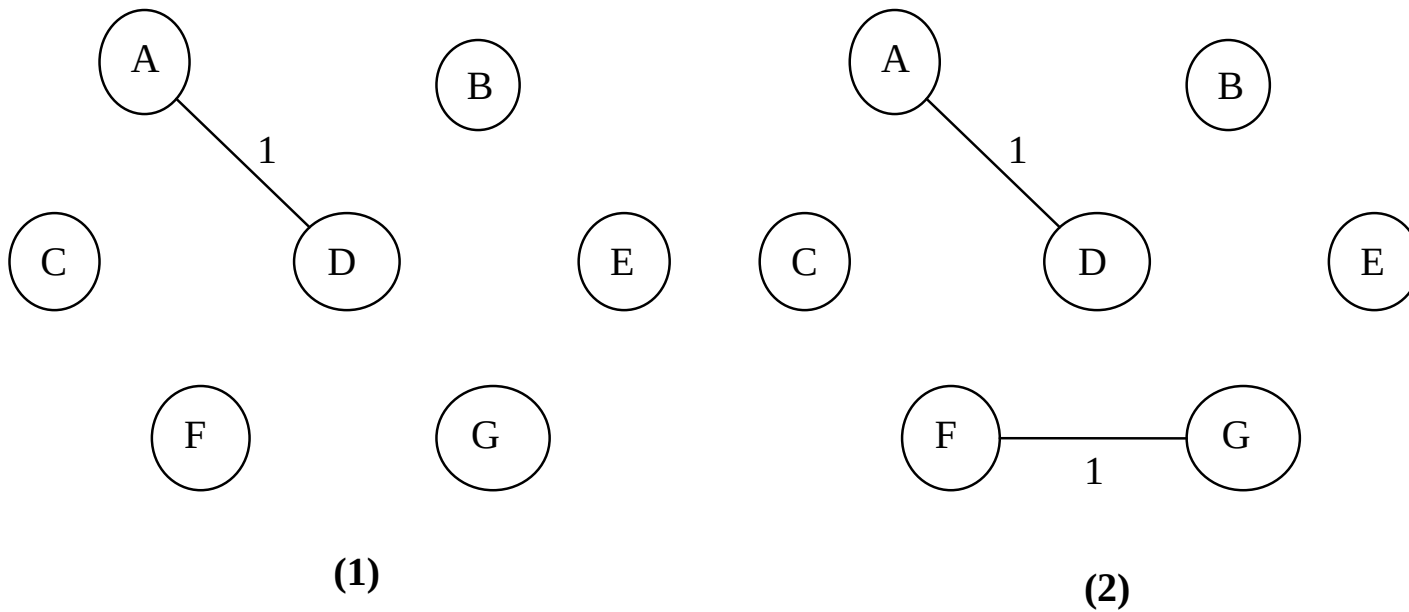
Step 2: Proceed sequentially to select one edge at a time joining n vertices of G such that no cycle is formed until $n-1$ edges are selected.

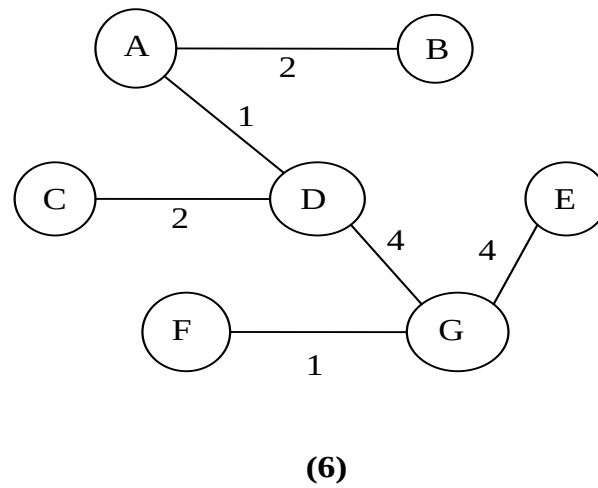
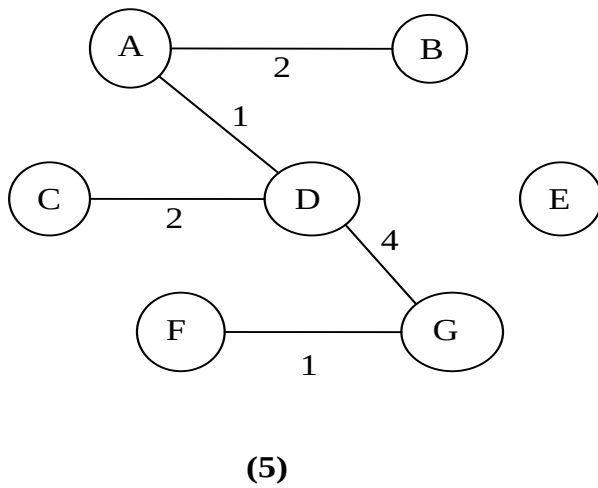
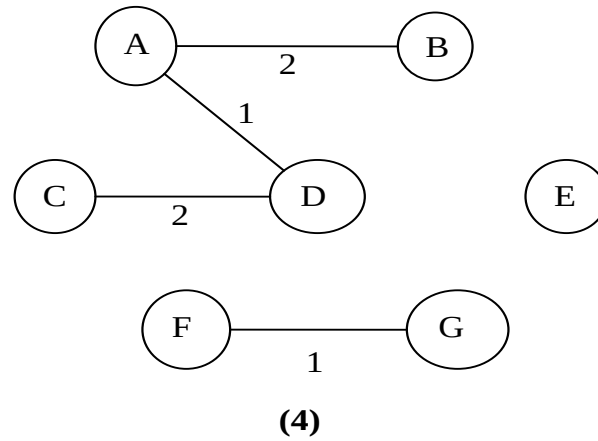
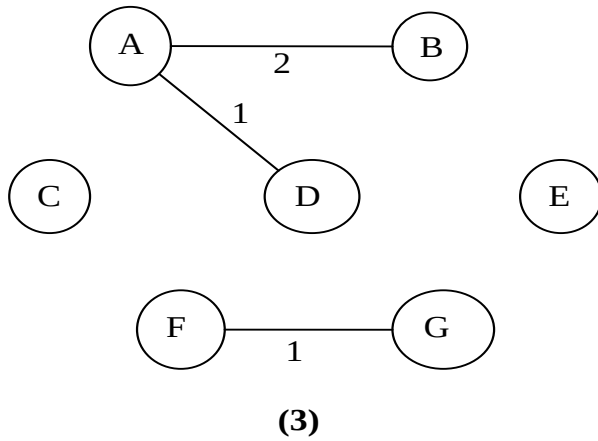
Step 3: Draw the $n-1$ edges that were selected forming a minimal spanning tree T of G .



We have,

Edges:	AD	FG	AB	CD	BD	AC	DG	CF	EG	DE	DF	BE
Weights	1	1	2	2	3	4	4	5	6	7	8	10





Hence, the minimum spanning tree is generated.